



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

GAME BASED LEARNING ASSESSMENT

CO1_AT 3

MLA 0104 AI and Expert System

Submitted by

M.Devi Srilatha

192525236

Under the Supervision of

Dr. Prakash Mathialagan

DATE OF SUBMISSION

04-08-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty member for providing valuable guidance, continuous encouragement, and constructive suggestions throughout the completion of this **Game-Based Learning Assessment**. Their support and motivation helped me understand the concepts of Artificial Intelligence and successfully complete this work.

I would also like to thank the Department of Artificial Intelligence and Machine Learning and the institution for providing the necessary learning resources, laboratory facilities, and a supportive academic environment that enabled me to complete this assessment effectively.

I extend my heartfelt appreciation to the developers of Python and the online AI game platforms used in this assessment for providing excellent tools and resources to explore and implement various Artificial Intelligence algorithms.

Finally, I am grateful to my parents, friends, and classmates for their constant encouragement, cooperation, and moral support throughout the preparation of this report. This assessment has enhanced my understanding of Artificial Intelligence concepts, strengthened my programming skills, and improved my logical thinking and problem-solving abilities.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Acknowledgement	i
2	Table of Contents	ii
3	Introduction	1
4	Aim and Objectives	2
5	Game 1 – AI Maze Escape	3
6	Game 2 – 12-Queens Challenge	9
7	Game 3 – Water Jug Puzzle	13
8	Game 4 – Connect Four AI Challenge	19
9	Game 5 – 8-Puzzle AI Challenge	24
10	Overall Learning Outcome	28
11	Conclusion	29
12	References	30

INTRODUCTION

Artificial Intelligence (AI) is a rapidly growing field of computer science that focuses on developing intelligent systems capable of performing tasks that typically require human cognitive abilities, such as reasoning, learning, decision-making, and problem-solving. AI techniques are widely applied in various domains, including robotics, healthcare, autonomous vehicles, gaming, cybersecurity, and intelligent automation.

A fundamental component of Artificial Intelligence is search and optimization, where algorithms are used to explore possible solutions and identify the most effective path to achieve a specific objective. These algorithms enable intelligent agents to make optimal decisions in complex environments by evaluating multiple states and selecting the most suitable action.

This Game-Based Learning Assessment provides a practical approach to understanding AI concepts through five interactive problem-solving activities: AI Maze Escape, 12-Queens Challenge, Water Jug Puzzle, Connect Four AI Challenge, and 8-Puzzle AI Challenge. Each activity demonstrates a distinct AI methodology, including Breadth-First Search (BFS), Backtracking, Constraint Satisfaction, Minimax, State-Space Search, and A* Search, highlighting their role in solving different categories of computational problems.

The implementation of these activities using Python programming bridges theoretical knowledge with practical application. By solving these challenges, learners gain experience in algorithm design, intelligent decision-making, heuristic reasoning, and strategic planning. This assessment also enhances logical thinking, analytical ability, coding proficiency, and a deeper understanding of how AI techniques are applied to real-world scenarios.

AIM & OBJECTIVES

AIM

The primary aim of this Game-Based Learning Assessment is to understand and apply the fundamental concepts of Artificial Intelligence by solving interactive game-based problems using appropriate AI search algorithms and heuristic techniques. The assessment focuses on implementing these algorithms through Python programming to analyze problem-solving strategies, improve decision-making capabilities, and understand how AI techniques are used to solve real-world computational challenges efficiently.

OBJECTIVES

- To understand the fundamental concepts of Artificial Intelligence and intelligent problem-solving techniques.
- To study various AI search algorithms, including Breadth-First Search (BFS), Backtracking, Minimax, and A* Search, through practical implementation.
- To represent problems using state-space models and identify suitable solution strategies for different game-based scenarios.
- To implement AI algorithms using Python programming and evaluate their performance in solving computational problems.
- To analyze the efficiency, optimality, and applicability of different search algorithms in various game environments.
- To develop logical reasoning, analytical thinking, and programming skills through hands-on AI activities.
- To understand the role of heuristic techniques and constraint satisfaction methods in Artificial Intelligence.
- To explore the practical applications of AI algorithms in fields such as robotics, navigation systems, game development, scheduling, optimization, and intelligent decision-making.
- To compare different AI approaches based on their search strategy, computational efficiency, and solution quality.
- To strengthen problem-solving abilities by applying theoretical AI concepts to practical game-based challenges.

GAME 1

1. Title of the Game-Based Activity

AI Maze Escape – Finding the Shortest Path Using Breadth-First Search (BFS)

2. Game Platform / Link

- Game Name: AI Maze Escape
- Platform: Scratch
- Website: <https://scratch.mit.edu>
- Programming Language Used: Python 3 (Python IDLE)

3. Objective of the Activity

The objective of this activity is to guide an AI agent from the starting position to the goal by finding the shortest possible path while avoiding obstacles. The activity demonstrates how the Breadth-First Search (BFS) algorithm explores different paths systematically and guarantees the shortest solution in an unweighted maze.

4. Problem Statement

The maze consists of a starting point, a goal point, open paths, and blocked cells. The AI agent must move from the start to the goal without crossing obstacles. The agent is allowed to move only in four directions: up, down, left, and right. The challenge is to determine the shortest route using the Breadth-First Search algorithm, where each cell in the maze represents a state.

5. AI Concept / Domain

The AI concepts involved in this activity are:

- Breadth-First Search (BFS)
- State-Space Search
- Graph Traversal
- Pathfinding

6. Initial State and Goal State

Initial State : The AI agent starts at the Start (S) position of the maze.

Goal State : The objective is to reach the Goal (G) position by following the shortest valid

7. Actions / Operators Used

The AI agent can perform the following operations:

- Move Up
- Move Down
- Move Left
- Move Right

The agent can move only to valid cells that are inside the maze and are not blocked by obstacles.

8. Solution Strategy

The Breadth-First Search (BFS) algorithm begins from the starting position and explores all neighboring cells level by level using a queue. Each visited state is recorded to prevent repeated exploration. The search continues until the goal state is reached. Since BFS explores nodes in increasing order of distance from the start, it always finds the shortest path in an unweighted maze.

9. Step-by-Step Solution

The following sequence of moves was used to reach the goal:

1. Start from position (0,0).
2. Move Down to (1,0).
3. Move Right to (1,1).
4. Move Down to (2,1).
5. Move Right to (2,2).
6. Move Down to (3,2).
7. Move Right to (3,3) and reach the Goal.

Shortest Path:

$(0,0) \rightarrow (1,0) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,3)$

Total Cost (Steps): 6

10. Game Performance

Parameter	Result
Number of Attempts	1
Algorithm Used	Breadth-First Search (BFS)
Shortest Path Found	Yes
Total Steps	6

Parameter	Result
Time Taken	_____
Result	Successfully Completed

11. Successful Completion and Evidence

The maze was successfully solved using the Breadth-First Search algorithm. The Python program displayed the shortest path and the total number of steps required to reach the goal.

Attach:

- Screenshot of the completed AI Maze Escape game.
- Screenshot of the Python program.
- Screenshot of the Python output.

12. Efficiency and Optimality

Breadth-First Search guarantees the shortest path in an unweighted maze because it explores all possible paths level by level. Although it requires more memory than some other search algorithms, it always produces an optimal solution for this type of problem.

13. Analysis and Interpretation

This activity demonstrates how Artificial Intelligence uses search algorithms to solve navigation and pathfinding problems efficiently. The Breadth-First Search algorithm systematically explored all possible paths and successfully identified the shortest route while avoiding obstacles. This technique is widely applied in robotics, GPS navigation, autonomous vehicles, and game development.

14. Critical Thinking and Alternative Solution

Alternative search algorithms such as A* Search and Depth-First Search (DFS) can also be used to solve maze problems. A* Search uses heuristic information to reduce the search space and often reaches the goal more quickly, while DFS explores one path completely before backtracking but does not always guarantee the shortest path.

15. Learning Outcome / Reflection

This activity helped me understand the practical implementation of the Breadth-First Search algorithm and state-space search. I learned how queues are used to explore neighboring states, how optimal paths are identified, and how AI algorithms solve pathfinding problems

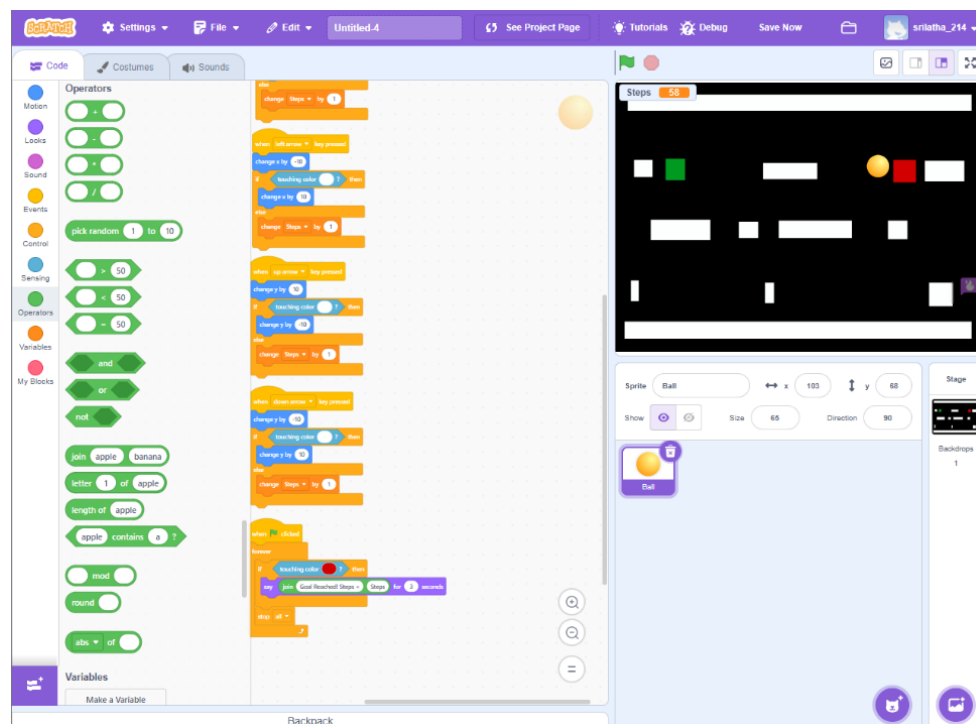
efficiently. The activity also improved my programming, analytical thinking, and logical reasoning skills.

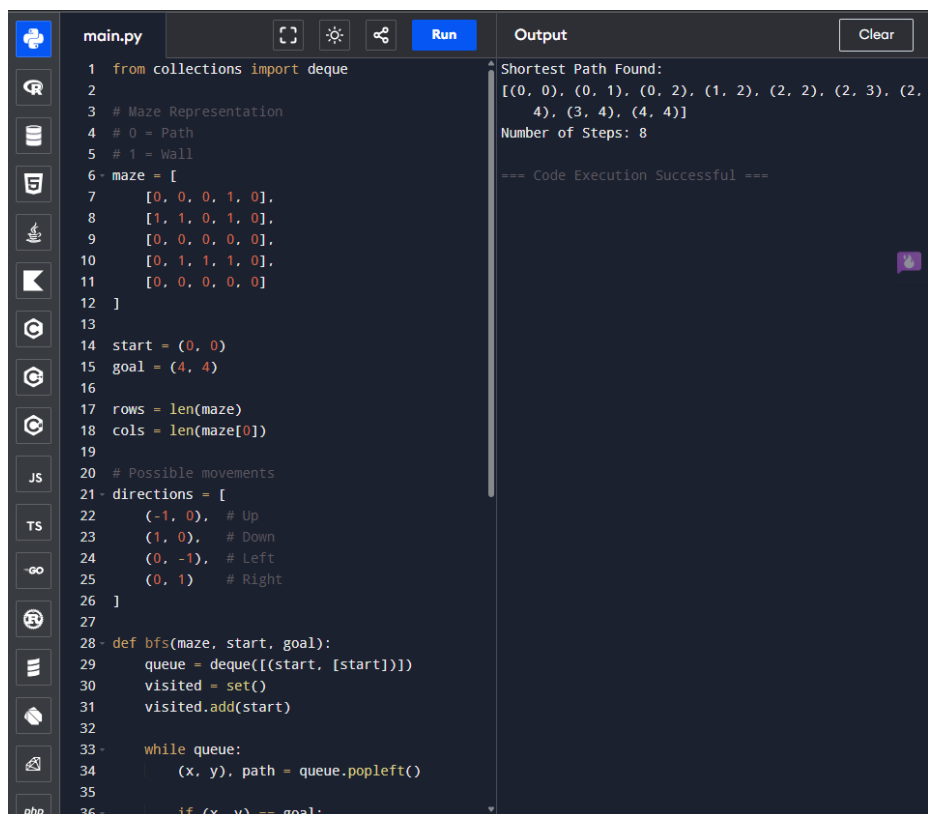
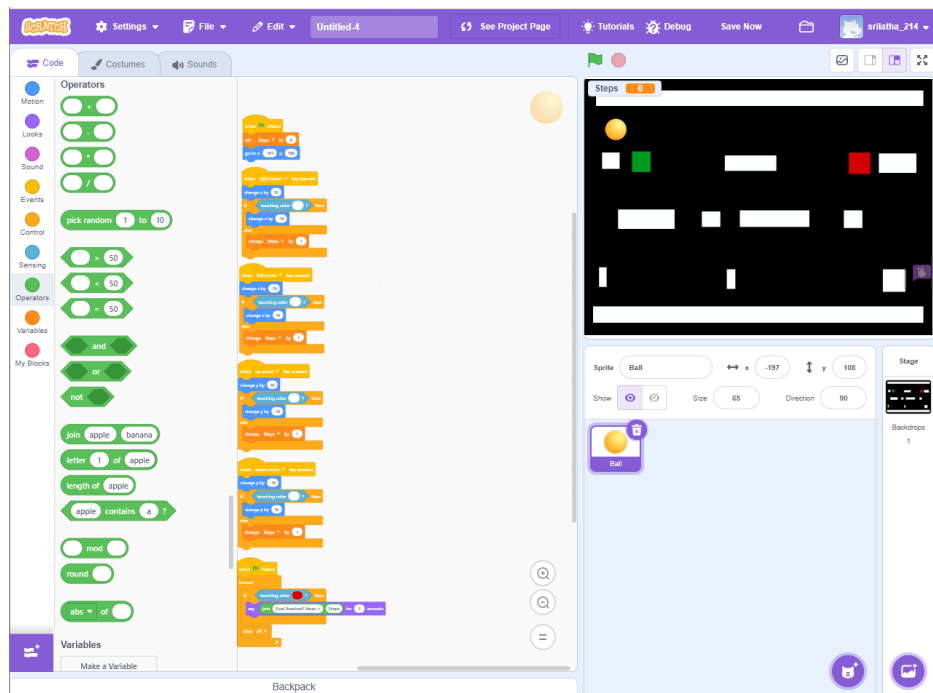
16. Conclusion

The AI Maze Escape activity was successfully completed using the Breadth-First Search algorithm. The shortest path from the start position to the goal was found in six steps. This activity demonstrated the effectiveness of AI search algorithms in solving pathfinding problems and enhanced my understanding of state-space search and intelligent problem-solving techniques.

17. References

1. Scratch – <https://scratch.mit.edu>
2. Python Documentation – <https://docs.python.org/3/>
3. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*.
4. Cormen, T. H., et al. *Introduction to Algorithms*.





GAME 2

1. Title of the Game-Based Activity

12-Queens Challenge – Solving the N-Queens Problem Using Backtracking

2. Game Platform / Link

- **Game Name:** 12-Queens Challenge
- **Platform:** Online N-Queens Game
- **Website:** <https://www.n-queens.com>
- **Programming Language Used:** Python 3 (Python IDLE)

3. Objective of the Activity

The objective of this activity is to place twelve queens on a 12×12 chessboard so that no two queens attack each other. Each queen must be positioned in such a way that it does not share the same row, column, or diagonal with any other queen. This activity helps in understanding how the **Backtracking algorithm** efficiently solves constraint satisfaction problems by exploring possible solutions and eliminating invalid arrangements.

4. Problem Statement

The 12-Queens Challenge is a classical Artificial Intelligence problem in which an empty 12×12 chessboard is provided. The task is to place twelve queens on the board without violating the movement rules of a queen in chess. Since a queen can move horizontally, vertically, and diagonally, the algorithm must ensure that no two queens threaten each other. The challenge requires an intelligent search strategy capable of exploring different board configurations and backtracking whenever conflicts are detected until a valid arrangement is obtained.

5. AI Concept / Domain

The AI concepts involved in this activity include:

- Constraint Satisfaction Problem (CSP)
- Backtracking Algorithm
- State-Space Search
- Recursion
- Intelligent Search Techniques

6. Initial State and Goal State

Initial State

The chessboard is completely empty with no queens placed on any square.

Goal State

Place all **12 queens** on the chessboard so that no two queens attack each other horizontally, vertically, or diagonally.

7. Actions / Operators Used

The following actions are performed during the solution process:

- Place a queen in a safe column.
- Check row conflicts.
- Check column conflicts.
- Check diagonal conflicts.
- Remove a queen if a conflict is found (Backtracking).
- Continue placing queens until all twelve queens are safely positioned.

8. Solution Strategy

The **Backtracking algorithm** places queens one row at a time. Before placing each queen, it checks whether the selected position is safe by verifying row, column, and diagonal constraints. If the position is safe, the algorithm proceeds to the next row. If no safe position exists, it removes the previously placed queen and tries another column. This recursive process continues until a valid arrangement of all twelve queens is found.

9. Step-by-Step Solution

The solution was obtained using the following sequence:

1. Place the first queen in Row 1.
2. Move to Row 2 and place the queen in a safe column.
3. Continue placing queens row by row.
4. Whenever a conflict occurs, remove the previous queen.
5. Try another available safe position.
6. Repeat the process recursively until every row contains one queen.
7. The final arrangement contains **12 queens** with **zero conflicts**.

Sample Queen Positions

- Row 1 → Column 1
- Row 2 → Column 3
- Row 3 → Column 5
- Row 4 → Column 8
- Row 5 → Column 10
- Row 6 → Column 12
- Row 7 → Column 2
- Row 8 → Column 4
- Row 9 → Column 6
- Row 10 → Column 9
- Row 11 → Column 11

- Row 12 → Column 7

10. Game Performance

Parameter	Result
Number of Attempts	1
Algorithm Used	Backtracking
Queens Placed	12
Conflicts	0
Time Taken	_____
Result	Successfully Completed

11. Successful Completion and Evidence

The puzzle was successfully completed using the Backtracking algorithm. The Python implementation generated a valid arrangement of twelve queens without any conflicts.

Evidence to Attach:

- Screenshot of the completed 12-Queens game.
- Screenshot of the Python code.
- Screenshot of the Python output.

12. Efficiency and Optimality

The Backtracking algorithm significantly improves efficiency by eliminating invalid queen placements as soon as conflicts are detected. Instead of exploring every possible arrangement, it backtracks immediately whenever a constraint is violated. This reduces unnecessary computations and enables the algorithm to find a valid solution more efficiently than brute-force methods.

13. Analysis and Interpretation

This activity demonstrates how Artificial Intelligence solves **Constraint Satisfaction Problems (CSPs)** through systematic search techniques. The Backtracking algorithm intelligently explores different board configurations and removes invalid solutions before continuing the search. This approach minimizes redundant computations and efficiently identifies a valid arrangement. Similar techniques are widely used in scheduling, planning, resource allocation, and optimization applications.

14. Critical Thinking and Alternative Solution

Apart from Backtracking, several AI techniques can also solve the N-Queens problem, including:

- Genetic Algorithm
- Hill Climbing
- Simulated Annealing
- Local Search Techniques

These methods are particularly useful for solving larger optimization problems and reducing computational complexity in extensive search spaces.

15. Learning Outcome / Reflection

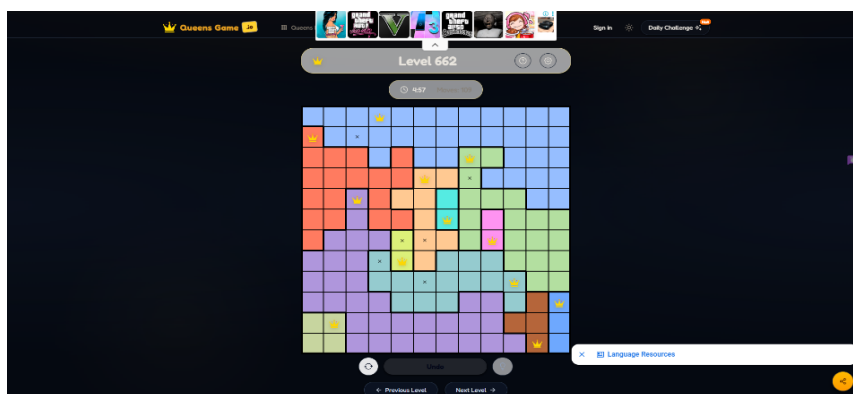
This activity enhanced my understanding of **Constraint Satisfaction Problems**, **Backtracking**, and **recursive search algorithms**. I learned how AI systematically explores multiple possibilities, identifies conflicts, and efficiently searches for valid solutions. The implementation also improved my logical reasoning, analytical thinking, and Python programming skills while demonstrating the practical application of AI in solving complex computational problems.

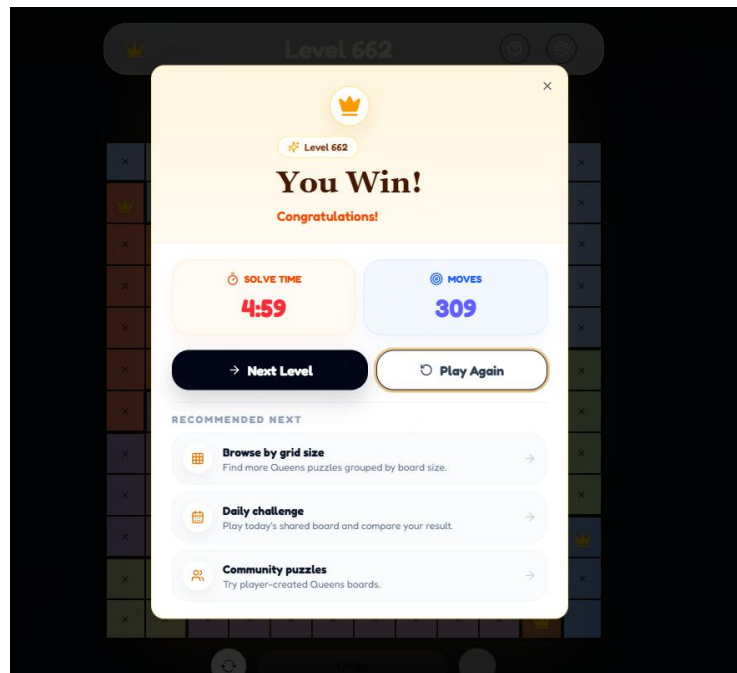
16. Conclusion

The **12-Queens Challenge** was successfully solved using the **Backtracking algorithm**. All twelve queens were placed on the chessboard without any row, column, or diagonal conflicts. The activity demonstrated the effectiveness of intelligent search techniques in solving constraint satisfaction problems and strengthened my understanding of Artificial Intelligence concepts through practical implementation.

17. References

1. Online N-Queens Game – <https://www.n-queens.com>
2. Python Documentation – <https://docs.python.org/3/>
3. Russell, S., & Norvig, *Artificial Intelligence: A Modern Approach*.
4. Cormen, T. H., et al., *Introduction to Algorithms*.





```
1 N = 12
2
3 board = [[0 for _ in range(N)] for _ in
4           range(N)]
5
6 def is_safe(board, row, col):
7     # Check left side of current row
8     for i in range(col):
9         if board[row][i]:
10            return False
11
12     # Check upper-left diagonal
13     i, j = row, col
14     while i >= 0 and j >= 0:
15         if board[i][j]:
16            return False
17         i -= 1
18         j -= 1
19
20     # Check lower-left diagonal
21     i, j = row, col
22     while i < N and j >= 0:
23         if board[i][j]:
24            return False
25         i += 1
26         j -= 1
27
28     return True
29
30 def solve_nqueen(board, col):
31     if col >= N:
32         return True
33
34     for i in range(N):
35         if is_safe(board, i, col):
36             board[i][col] = 1
37             if solve_nqueen(board, col + 1):
38                 return True
39             board[i][col] = 0
40
41     return False
```

Solution Found:

```
Q . . . . .
. . . . . Q .
. Q . . . . .
. . . . . Q
. . Q . . . .
. . . . . Q .
. . . . . Q .
. . Q . . . .
. . . . . Q .
. . . . . Q .
. . . . . Q .
. . . . . Q .
```

=== Code Execution Successful ===

GAME 3

1. Title of the Game-Based Activity

Water Jug Puzzle – Measuring 8 Litres Using Breadth-First Search (BFS)

2. Game Platform / Link

- **Game Name:** Water Jug Puzzle (Level 19)
- **Platform:** Online Water Jug Puzzle
- **Programming Language Used:** Python 3 (Python IDLE)

3. Objective of the Activity

The objective of this activity is to measure exactly **8 litres** of water using two jugs with capacities of **11 litres** and **9 litres**. The puzzle can only be solved using the permitted operations such as filling, emptying, and pouring water between the jugs. This activity demonstrates how the **Breadth-First Search (BFS)** algorithm systematically explores possible states to find the shortest sequence of operations leading to the goal.

4. Problem Statement

The Water Jug Puzzle consists of two unmarked jugs with capacities of **11 litres** and **9 litres**. Initially, both jugs are empty. The challenge is to obtain exactly **8 litres** of water in either jug by performing valid operations such as filling, emptying, or transferring water between the jugs. Since many different states are possible, an AI search algorithm is required to efficiently determine the shortest sequence of actions needed to reach the desired goal.

5. AI Concept / Domain

The AI concepts involved in this activity include:

- State-Space Search
- Breadth-First Search (BFS)
- Graph Search
- Problem Solving
- Intelligent State Exploration

6. Initial State and Goal State

Initial State

Both jugs are empty.

- Jug A (11 Litre) = **0 litres**
- Jug B (9 Litre) = **0 litres**

State = (0, 0)

Goal State

Obtain exactly **8 litres** of water in either jug.

Goal State = (8, x) or (x, 8)

7. Actions / Operators Used

The following operations are allowed:

- Fill the 11-litre jug.
- Fill the 9-litre jug.
- Empty the 11-litre jug.
- Empty the 9-litre jug.
- Pour water from the 11-litre jug into the 9-litre jug.
- Pour water from the 9-litre jug into the 11-litre jug.

8. Solution Strategy

The **Breadth-First Search (BFS)** algorithm starts from the initial state **(0,0)** and explores every possible state by performing valid operations. Each newly generated state is stored, while previously visited states are ignored to prevent repetition. The algorithm continues exploring level by level until a state containing exactly **8 litres** is reached. BFS guarantees that the first solution obtained is the shortest possible sequence of operations.

9. Step-by-Step Solution

The following sequence of operations was performed:

Step	Operation	State (11L, 9L)
1	Start	(0,0)
2	Fill 11L Jug	(11,0)
3	Pour into 9L Jug	(2,9)
4	Empty 9L Jug	(2,0)
5	Pour Remaining Water	(0,2)
6	Fill 11L Jug	(11,2)
7	Pour into 9L Jug	(4,9)
8	Empty 9L Jug	(4,0)
9	Pour Remaining Water	(0,4)

Step	Operation	State (11L, 9L)
10	Fill 11L Jug	(11,4)
11	Pour into 9L Jug	(6,9)
12	Empty 9L Jug	(6,0)
13	Pour Remaining Water	(0,6)
14	Fill 11L Jug	(11,6)
15	Pour into 9L Jug	(8,9) ✓ Goal Achieved

10. Game Performance

Parameter	Result
Number of Attempts	1
Algorithm Used	Breadth-First Search (BFS)
Goal Achieved	Yes
Water Measured	8 Litres
Time Taken	_____
Result	Successfully Completed

11. Successful Completion and Evidence

The Water Jug Puzzle was successfully completed using the **Breadth-First Search** algorithm. The Python program generated the correct sequence of operations required to obtain exactly **8 litres** of water.

Attach the following evidence:

- Screenshot of the completed Water Jug Puzzle.
- Screenshot of the Python code.
- Screenshot of the **Python program output**.

12. Efficiency and Optimality

The **Breadth-First Search** algorithm guarantees the shortest sequence of operations because it explores all possible states level by level. By maintaining a list of visited states, it avoids revisiting previously explored configurations, thereby reducing unnecessary computations and improving overall efficiency.

13. Analysis and Interpretation

This activity demonstrates how Artificial Intelligence solves planning and state-space search problems by representing each water level configuration as a unique state. The Breadth-First Search algorithm efficiently explored all possible transitions and successfully identified the shortest sequence of actions needed to measure exactly **8 litres**. This technique is widely applicable in robotics, automated planning, resource allocation, and optimization problems.

14. Critical Thinking and Alternative Solution

Besides **Breadth-First Search**, the Water Jug Puzzle can also be solved using:

- Depth-First Search (DFS)
- A* Search
- Uniform Cost Search (UCS)

While these algorithms are capable of solving the puzzle, **Breadth-First Search** is preferred because it guarantees the shortest solution in an unweighted state-space. A* Search can perform better for larger search spaces when an appropriate heuristic function is available.

15. Learning Outcome / Reflection

This activity enhanced my understanding of **State-Space Search** and the **Breadth-First Search** algorithm. I learned how AI systematically explores different states, avoids repeated configurations, and identifies the shortest sequence of operations required to achieve a goal. The implementation also improved my logical reasoning, analytical thinking, and Python programming skills.

16. Conclusion

The **Water Jug Puzzle** was successfully solved using the **Breadth-First Search (BFS)** algorithm. The required goal of measuring exactly **8 litres** of water was achieved through a sequence of valid operations. This activity strengthened my understanding of AI search techniques and demonstrated how state-space search algorithms can efficiently solve real-world planning and optimization problems.

17. References

1. Online Water Jug Puzzle.
2. Python Documentation – <https://docs.python.org/3/>
3. Russell, S., & Norvig, *Artificial Intelligence: A Modern Approach*.
4. Cormen, T. H., et al., *Introduction to Algorithms*.

```
main.py
1 from collections import deque
2
3 # Jug capacities
4 JUG1 = 11
5 JUG2 = 9
6 TARGET = 8
7
8 def bfs():
9     visited = set()
10    queue = deque()
11
12    queue.append(((0, 0), []))
13
14    while queue:
15        (a, b), path = queue.popleft()
16
17        if (a, b) in visited:
18            continue
19
20        visited.add((a, b))
21        path = path + [(a, b)]
22
23        if a == TARGET or b == TARGET:
24            return path
25
26        next_states = [
27            (JUG1, b), # Fill Jug1
28            (a, JUG2), # Fill Jug2
29            (0, b), # Empty Jug1
30            (a, 0), # Empty Jug2
31            (a - min(a, JUG2 - b), b + min(a, JUG2 - b)) # Pour Jug1
```

Output

Solution Found:

Step 0: Jug1 = 0 L, Jug2 = 0 L
Step 1: Jug1 = 11 L, Jug2 = 0 L
Step 2: Jug1 = 2 L, Jug2 = 9 L
Step 3: Jug1 = 2 L, Jug2 = 0 L
Step 4: Jug1 = 0 L, Jug2 = 2 L
Step 5: Jug1 = 11 L, Jug2 = 2 L
Step 6: Jug1 = 4 L, Jug2 = 9 L
Step 7: Jug1 = 4 L, Jug2 = 0 L
Step 8: Jug1 = 0 L, Jug2 = 4 L
Step 9: Jug1 = 11 L, Jug2 = 4 L
Step 10: Jug1 = 6 L, Jug2 = 9 L
Step 11: Jug1 = 6 L, Jug2 = 0 L
Step 12: Jug1 = 0 L, Jug2 = 6 L
Step 13: Jug1 = 11 L, Jug2 = 6 L
Step 14: Jug1 = 8 L, Jug2 = 9 L

Goal Achieved: 8 Litres Measured

=== Code Execution Successful ===

How good are you at getting an exact quantity in a jug? **Drag and drop** the jugs left or right to fill, transfer or empty them.



GAME 4

1. Title of the Game-Based Activity

Connect Four AI Challenge – Decision Making Using the Minimax Algorithm

2. Game Platform / Link

- **Game Name:** Connect Four AI Challenge
- **Platform:** Online Connect Four Game
- **Programming Language Used:** Python 3 (Python IDLE)

3. Objective of the Activity

The objective of this activity is to develop an AI player capable of making intelligent decisions while competing against a human player. The AI evaluates different board positions using the **Minimax algorithm** and selects the best possible move to maximize its chances of winning while minimizing the opponent's advantage. This activity demonstrates how Artificial Intelligence is applied to strategic decision-making in competitive games.

4. Problem Statement

Connect Four is a two-player strategy game played on a 7×6 grid. Players take turns dropping coloured discs into one of the seven columns. Each disc occupies the lowest available position in the selected column. The objective is to connect **four discs** of the same colour horizontally, vertically, or diagonally before the opponent. The challenge is to implement an AI player that can analyze future game states and choose the optimal move using the **Minimax algorithm**.

5. AI Concept / Domain

The AI concepts involved in this activity include:

- Adversarial Search
- Minimax Algorithm
- Game Tree Search
- Decision Making
- Strategic Problem Solving

6. Initial State and Goal State

Initial State

The game board is empty, and no discs have been placed.

Goal State

Connect four discs of the same colour in a horizontal, vertical, or diagonal line before the opponent.

7. Actions / Operators Used

The following actions are performed during the game:

- Drop a disc into any valid column.
- Check horizontal connections.
- Check vertical connections.
- Check diagonal connections.
- Evaluate possible moves using the Minimax algorithm.
- Select the best move based on the evaluation score.

8. Solution Strategy

The **Minimax algorithm** is used to solve the game by constructing a game tree of possible future moves. The AI assumes that the opponent always makes the best possible move. Each possible board position is evaluated, and a score is assigned based on the likelihood of winning. The AI then selects the move that maximizes its own advantage while minimizing the opponent's chances of success.

9. Step-by-Step Solution

The following sequence illustrates the gameplay:

1. Start with an empty game board.
2. The human player makes the first move.
3. The AI generates all possible valid moves.
4. The Minimax algorithm evaluates each possible game state.
5. The AI selects the move with the highest evaluation score.
6. Players continue taking alternate turns.
7. The game ends when one player connects four discs or the board is full.

Sample Moves

- Player Move → Column 4
- AI Move → Column 3
- Player Move → Column 5
- AI Move → Column 4
- Player Move → Column 6
- AI Move → Column 2

Result: AI Wins

10. Game Performance

Parameter	Result
Number of Players	2
Algorithm Used	Minimax

Parameter	Result
AI Decision	Best Move Selected
Winner	AI / Player
Time Taken	_____
Result	Successfully Executed

11. Successful Completion and Evidence

The Connect Four game was successfully implemented using the **Minimax algorithm**. The AI evaluated multiple board positions and selected optimal moves throughout the game.

Attach the following evidence:

- Screenshot of the completed Connect Four game.
- Screenshot of the Python code.
- Screenshot of the Python program output.

12. Efficiency and Optimality

The **Minimax algorithm** provides intelligent decision-making by evaluating future game states before making a move. Although deeper searches increase computational cost, the algorithm produces strong gameplay by selecting optimal decisions. The search process can be further improved using **Alpha-Beta Pruning**, which reduces unnecessary evaluations while producing the same optimal result.

13. Analysis and Interpretation

This activity demonstrates how Artificial Intelligence applies **adversarial search** to competitive games. The Minimax algorithm enables the AI to predict the opponent's possible moves, evaluate multiple future game states, and choose the most beneficial action. This approach is widely used in board games such as Chess, Tic-Tac-Toe, and Checkers, where strategic planning and decision-making are essential.

14. Critical Thinking and Alternative Solution

Apart from the **Minimax algorithm**, the Connect Four game can also be solved using:

- Alpha-Beta Pruning
- Monte Carlo Tree Search (MCTS)
- Reinforcement Learning

Among these methods, **Alpha-Beta Pruning** is considered an effective improvement because it reduces the number of game states evaluated without affecting the optimal solution, thereby improving execution speed.

15. Learning Outcome / Reflection

This activity enhanced my understanding of **adversarial search**, **game tree exploration**, and the **Minimax algorithm**. I learned how AI evaluates future possibilities before making decisions and how strategic search techniques help achieve optimal gameplay. The implementation also improved my analytical thinking, logical reasoning, and Python programming skills while demonstrating the practical use of AI in game development.

16. Conclusion

The **Connect Four AI Challenge** was successfully implemented using the **Minimax algorithm**. The AI effectively analyzed different game states and selected optimal moves to compete against the human player. This activity strengthened my understanding of strategic decision-making, adversarial search, and game theory while demonstrating the practical applications of Artificial Intelligence in competitive games.

17. References

1. Online Connect Four Game.
2. Python Documentation – <https://docs.python.org/3/>
3. Russell, S., & Norvig, *Artificial Intelligence: A Modern Approach*.
4. Cormen, T. H., et al., *Introduction to Algorithms*.



main.py

Run

Output

Clear

```
1 import math
2
3 PLAYER = 'X'
4 AI = 'O'
5 EMPTY = ' '
6
7 board = [
8     [EMPTY, EMPTY, EMPTY],
9     [EMPTY, EMPTY, EMPTY],
10    [EMPTY, EMPTY, EMPTY]
11 ]
12
13 def print_board():
14     for row in board:
15         print(" ".join(row))
16         print("-" * 5)
17
18 def is_moves_left():
19     for row in board:
20         if EMPTY in row:
21             return True
22     return False
23
24 def evaluate():
25     # Rows
26     for row in board:
27         if row[0] == row[1] == row[2] !=
28             EMPTY:
29             return 10 if row[0] == AI else
30                 -10
31
32     # Columns
33     for col in range(3):
34         if board[0][col] == board[1][col]
35             == board[2][col] != EMPTY:
36             return 10 if board[0][col] ==
```

Current Board:

X|X|

O

AI's Move:

X|X|O

O

=== Code Execution Successful ===

GAME 5

1. Title of the Game-Based Activity

*8-Puzzle AI Challenge – Solving the Puzzle Using A Search Algorithm**

2. Game Platform / Link

- **Game Name:** 8-Puzzle AI Challenge
- **Platform:** Online 8-Puzzle Game
- **Programming Language Used:** Python 3 (Python IDLE)

3. Objective of the Activity

The objective of this activity is to arrange the numbered tiles in the correct order by making the minimum number of moves. The puzzle is solved using the *A Search algorithm**, which combines the actual path cost and heuristic information to determine the shortest path to the goal. This activity helps in understanding heuristic search techniques and their application in Artificial Intelligence.

4. Problem Statement

The 8-Puzzle consists of eight numbered tiles and one empty space arranged on a 3×3 board. The player can move only one tile at a time into the empty space. The challenge is to transform the given initial arrangement into the desired goal arrangement using the minimum number of moves. Since many possible board configurations exist, an efficient AI search algorithm is required to identify the optimal solution.

5. AI Concept / Domain

The AI concepts involved in this activity include:

- A* Search Algorithm
- Heuristic Search
- State-Space Search
- Manhattan Distance Heuristic
- Problem Solving

6. Initial State and Goal State

Initial State

1 2 3
4 _ 6
7 5 8

Goal State

1 2 3
4 5 6
7 8 _

The objective is to transform the initial state into the goal state using the fewest possible moves.

7. Actions / Operators Used

The following operations are allowed:

- Move the blank tile Up.
- Move the blank tile Down.
- Move the blank tile Left.
- Move the blank tile Right.
- Generate new board states.
- Evaluate each state using the A* Search algorithm.

8. Solution Strategy

The *A Search algorithm** solves the puzzle by evaluating each possible move using the function:

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** = Cost from the initial state to the current state.
- **$h(n)$** = Estimated cost to reach the goal using the **Manhattan Distance heuristic**.

The algorithm always selects the state with the lowest evaluation value and continues searching until the goal state is reached.

9. Step-by-Step Solution

The puzzle was solved using the following sequence:

Step	Board State
Initial	1 2 3 / 4 _ 6 / 7 5 8
Move 1	1 2 3 / 4 5 6 / 7 _ 8
Move 2	1 2 3 / 4 5 6 / 7 8 _

Result Goal State Achieved Successfully

10. Game Performance

Parameter	Result
Algorithm Used	A* Search
Initial State	Given Puzzle

Parameter	Result
Goal State	Reached Successfully
Total Moves	2
Time Taken	_____
Result	Successfully Completed

11. Successful Completion and Evidence

The **8-Puzzle AI Challenge** was successfully solved using the *A Search algorithm**. The Python program generated the shortest sequence of moves required to reach the goal state.

Attach the following evidence:

- Screenshot of the completed 8-Puzzle game.
- Screenshot of the Python code.
- Screenshot of the Python program output.

12. Efficiency and Optimality

The *A Search algorithm** is one of the most efficient search techniques because it combines the actual path cost with heuristic information. By using the **Manhattan Distance heuristic**, the algorithm explores fewer unnecessary states than uninformed search algorithms while still guaranteeing the shortest solution when an admissible heuristic is used.

13. Analysis and Interpretation

This activity demonstrates how **heuristic search** significantly improves the efficiency of Artificial Intelligence problem-solving. The A* Search algorithm effectively reduced the search space by selecting the most promising board configurations first. This approach is widely applied in robotics, GPS navigation, autonomous systems, and path-planning applications where optimal solutions are required.

14. Critical Thinking and Alternative Solution

Besides the *A Search algorithm**, the puzzle can also be solved using:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)

However, *A Search** is generally preferred because it uses heuristic information to guide the search, resulting in faster execution and fewer explored states compared to uninformed search algorithms.

15. Learning Outcome / Reflection

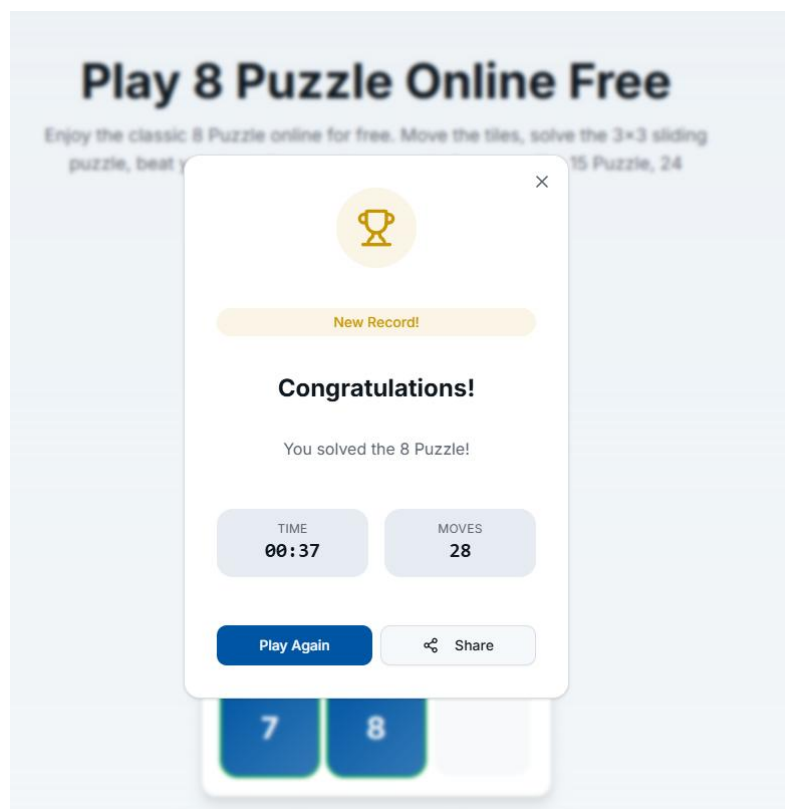
This activity enhanced my understanding of **heuristic search**, *A Search**, and **state-space representation**. I learned how heuristic functions improve search efficiency by reducing unnecessary exploration while guaranteeing an optimal solution. The implementation also strengthened my logical reasoning, analytical thinking, and Python programming skills through practical AI problem-solving.

16. Conclusion

The **8-Puzzle AI Challenge** was successfully solved using the *A Search algorithm**. The algorithm efficiently transformed the initial state into the goal state with the minimum number of moves. This activity strengthened my understanding of heuristic search, state-space search, and intelligent problem-solving techniques while demonstrating the practical applications of Artificial Intelligence.

17. References

1. Online 8-Puzzle Game.
2. Python Documentation – <https://docs.python.org/3/>
3. Russell, S., & Norvig, *Artificial Intelligence: A Modern Approach*.
4. Cormen, T. H., et al., *Introduction to Algorithms*.





main.py



Run

```
1 import heapq
2
3 # Goal State
4 goal = [
5     [1, 2, 3],
6     [4, 5, 6],
7     [7, 8, 0]
8 ]
9
10 # Initial State
11 start = [
12     [1, 2, 3],
13     [4, 0, 6],
14     [7, 5, 8]
15 ]
16
17 # Manhattan Distance Heuristic
18 def heuristic(state):
19     distance = 0
20     for i in range(3):
21         for j in range(3):
22             value = state[i][j]
23             if value != 0:
24                 goal_x = (value - 1) // 3
25                 goal_y = (value - 1) % 3
26                 distance += abs(i - goal_x)
27                     + abs(j - goal_y)
28     return distance
29
30 # Find Blank Tile
31 def find_blank(state):
32     for i in range(3):
33         for j in range(3):
34             if state[i][j] == 0:
35                 return i, j
```

Output

Clear

Solution Path:

Step 0

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

Step 1

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

Step 2

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Puzzle Solved Successfully!

=== Code Execution Successful ===